1. Compunere funcții cu mai mult de 1 parametru (ex: length . (++))

- Semnături funcții:
  - (.) :: (b -> c) -> (a -> b) -> a -> c
  - length :: Foldable t => t a -> Int
  - (++) :: [a] -> [a] -> [a]
- length . (++) => încearcă să găsească 3 tipuri a', b', c' a.î.
  - length :: b' -> c'
  - (++) :: a' -> b'
- **-> asociază la dreapta** deci pentru (++), o să ia:
  - a' = [a]
  - b' = [a] -> [a]
- Și de aici pică pentru că se așteaptă ca length să aibă tipul length :: b' -> c' = [a] -> [a] -> c' iar e de fapt length :: t a -> Int
- Eroarea este **Couldn't match expected type `[a1] -> t' with actual type `Int'** de la comparatia dintre [a] -> c' (notata in eroare ca [a1] -> t) si Int
- Ca să scăpați de eroare puteți să faceți funcția **uncurry** și să dați parametri sub formă de **pereche/tuplu**: **(length . (uncurry (++))) ([1, 2], [3, 4])**

2. Precedenta si asociativitate

- Aplicarea unei funcții are precedenta 10 ("operator" = spatiu)
- Toate celelalte de la 0 la 9
- Funcția cu prioritatea cea mai mare se aplica prima

Prec-edence	Left associative operators	Non-associative operators	Right associative operators
9	!!		.
8			^, ^^, **
7	*, /, `div`,		
	`mod`, `rem`, `quot`		
6	+, -		
5			:, ++
4		==, /=, <, <=, >, >=,	
		`elem`, `notElem`	
3			&&
2			||
1	>>, >>=		
0			$, $!, `seq`

Exemple asociativitate

1. **Stanga**
   a. _ (spatiu): f g h x = (((f g) h) x)
   b. + (adunare): f + g + h = (f + g) + h
2. **Dreapta**
   a. $ (operator de aplicare): f $ g $ h x = f (g (h x))
   b. . (compunere): (f . g . h) x = (f . (g . h)) x, dar f . g . h x = f . (g . (h x)) (aplicarea lui h are precedenta mai mare decat compunerea)
   c. : (adaugarea lui element la o lista): f : g : h : x = f : (g : (h : x))
   d. && (si logic): f && g && h = f && (g && h)

Exemple precedenta

1. f a b c d = a b + c d = (a b) + (c d)
   - _ prioritate 10, + prioritate 6
2. f x y z = x . y z = x . (y z)
   - _ prioritate 10, . prioritate 9
3. f x y z = x y . z = (x y) . z
   - _ prioritate 10, . prioritate 9
4. f x = (. map x) = (. (map x))
   - _ prioritate 10, . prioritate 9
5. f x y z = filter z . y $ x = ((filter z) . y)(x)
   - _ prioritate 10, . prioritate 9, $ prioritate 0

6. f x y z t = x ++ (y . z) t = x ++ ((y . z) t)
    - _ prioritate 10, . prioritate 9, ++ prioritate 5
    - Daca era fara paranteze: x ++ y . z t = x ++ (y . (z t)) si da eroare pentru ca incearca sa faca append intre o lista (x) si o functie (y . (z t))